

Two Julia Implementations of the Trigger/Target Particle Filter: A Comparison

Prepared for the `PartiallyObservedMarkovProcesses.jl` project

September 8, 2026

Abstract

Both repositories implement the same generalized sequential Monte Carlo (particle filter) recursion for a partially observed Markov process: resampling is triggered by a threshold on the effective sample size, and, when it occurs, is tempered by a power β that interpolates between equally weighted resampling and an untouched weighted particle representation. This note documents four places where Aaron King’s Julia translation (`PartiallyObservedMarkovProcesses.jl/aarons/src/pfilter.jl`) and Debsurya’s Julia translation (`PartiallyObservedMarkovProcesses.jl/src/pfilter.jl`) diverge, despite sharing the same trigger/target interface and the same recursive structure: an indexing error in how retained weights are carried through resampling, an unweighted terminal ancestry draw, and the presence or absence of a per-step resampling diagnostic. It also records, at length, a fourth apparent divergence — a supposed missing normalizing-constant credit at partial resampling — which on numerical examination turned out not to be a defect at all, and which has since been reverted; the argument that dismissed it is of more interest than the other three findings combined. It closes with the resulting unification of `wpfilter` as a thin wrapper around `pfilter`.

1 The trigger/target resampling scheme

Both implementations run the same filtering recursion. At each observation time, particles $x_1, \dots, x_n$ carry linear weights $w_1, \dots, w_n$, normalized so that $\bar{w} = n^{-1} \sum_j w_j = 1$. The effective sample size

$$\text{ESS} = \frac{\left(\sum_j w_j\right)^2}{\sum_j w_j^2}$$

is compared against a threshold set by the `trigger` parameter, $\text{trigger} \in [0, 1]$: resampling occurs whenever $\text{ESS} \leq \text{trigger} \cdot n$. `trigger`=1 resamples at every observation time (the classical bootstrap filter); `trigger`=0 never resamples, and the incremental weights are simply carried forward, multiplied into the running weight vector at the next observation time.

When resampling does occur, it is tempered by the `target` parameter $\beta \in [0, 1]$: particle j is selected as an ancestor with probability proportional to $w_j^{1-\beta}$, and each selected particle retains, going forward, a weight proportional to w_j^β (the weight of the particular ancestor selected), renormalized to unit mean across the new particle set. This is a genuine one-parameter family: at $\beta = 0$, ancestors are drawn proportional to the full weight w_j and every survivor is reweighted to 1 — the standard bootstrap resampling step. At $\beta = 1$, ancestors are drawn without regard to weight ($w_j^0 = 1$ for all j), and each survivor retains its ancestor’s original weight $w_j^1 = w_j$ — since uniform systematic selection of n ancestors from n slots reproduces the identity permutation up to a vanishing boundary effect, this carries the full weighted particle representation forward essentially unchanged. Both repositories implement this design; the divergences below concern its realization in code, not the design itself.

2 A positional-indexing error in weight retention

Once ancestor indices $p(1), \dots, p(n)$ have been drawn from the cumulative sums of $w_j^{1-\beta}$, the retained weight w_j^β must travel *with the particle it came from*: since $w_j = w_j^{1-\beta} \cdot w_j^\beta$ and the first factor has already been consumed

by the selection probability, the second factor is what remains to be assigned to the particle now occupying position $k = p^{-1}(j)$ in the resampled array.

Aaron’s Julia `systematic_resample!` instead raises the entire weight vector to the power β *in place*, in its *original positional order*, after the ancestor indices have already been drawn:

Listing 1: Aaron’s `systematic_resample!` (PartiallyObservedMarkovProcesses.jl_aarons/src/pfilter.jl, lines 336–363)

```
systematic_resample!(
    p::AbstractArray{I,1},
    w::AbstractArray{W,1},
    ucum::AbstractArray{W,1},
    beta::Float64,
) where {I,W} = begin
    s::W = 0
    alpha = 1-beta
    @inbounds for j in eachindex(w)
        s += w[j]^alpha
        ucum[j] = s
    end
    n::I = length(w)
    i::I = 1
    du::W = s/n
    u::W = -du*rand(W)
    @inbounds for j in eachindex(p)
        u += du
        while (u > ucum[i] && i < n)
            i += 1
        end
        p[j] = i
    end
    w .^= beta
    w ./= mean(w)
    nothing
end
```

The particle *states* are copied correctly downstream (`xf .= xp[p]`), so state k correctly becomes the state of ancestor $p(k)$, but the weight vector `w` is exponentiated positionally: the new weight at position k is w_k^β — the β -power of whatever particle *used to occupy* position k — not $w_{p(k)}^\beta$, the β -power of the ancestor actually selected. Whenever the resampling permutation p moves mass between positions, which is the norm rather than the exception, the state and its retained weight come from two different particles.

Aaron’s R implementation of the same routine (the C code behind `wpfilter` in the R `pomp` package) does not have this defect: it copies the retained weight of the selected ancestor directly, so the weight is indexed by the ancestor, not by position:

Listing 2: R `wpfilter.c`, inside the ancestor-selection loop, as reported by Debsurya

```
wt[k] = ws[sp];      /* ws = carried weights, sp = selected ancestor index */
```

Debsurya’s Julia translation follows the R form rather than Aaron’s Julia form:

Listing 3: Debsurya’s `systematic_resample!` (PartiallyObservedMarkovProcesses.jl/src/pfilter.jl, lines 448–481), retained-weight step

```
@inbounds for j in eachindex(p)
    ucum[j] = w[p[j]]^beta      # retained weight of the selected ancestor
end
@inbounds for j in eachindex(w)
    w[j] = ucum[j]
end
m::W = mean(w)
w ./= m
log(m*s/n)
```

Here `w[p[j]]` is evaluated *before* `w` is overwritten, using `ucum` as scratch space, so each new weight is correctly indexed by the selected ancestor. At $\beta = 0$ the two forms coincide trivially (every weight is 1 regardless of index), which is presumably why the error has not surfaced in the classical (target = 0) filter; it is only visible for $\beta > 0$.

3 The mass removed at partial resampling: an apparent defect that is not one

Write $w_1, \dots, w_n$ for the unit-mean weights immediately prior to resampling, and $S' = \sum_j w_j^{1-\beta}$ for the normalizing sum of the selection probabilities $\pi_j = w_j^{1-\beta}/S'$. A short importance-sampling argument shows that the *properly weighted* particle representation after resampling — the one whose weighted empirical measure is unbiased for the same target as before resampling — assigns the particle selected at position k (ancestor $j = p(k)$) the weight

$$w'_k = w_j^\beta \cdot \frac{S'}{n},$$

not simply w_j^β . Indeed, for any test function f ,

$$\mathbb{E}_{p(k) \sim \pi} [w'_k f(x_{p(k)})] = \sum_j \pi_j w_j^\beta \frac{S'}{n} f(x_j) = \sum_j \frac{w_j^{1-\beta}}{S'} w_j^\beta \frac{S'}{n} f(x_j) = \frac{1}{n} \sum_j w_j f(x_j),$$

reproducing the pre-resampling weighted average exactly, in expectation over the resampling randomness, for every β .

Both implementations instead renormalize the retained weights to unit mean, since every other routine in the filter (the effective-sample-size and conditional-log-likelihood computation in particular) assumes unit-mean weights on entry. Writing $m = n^{-1} \sum_k w_{p(k)}^\beta$ for the mean of the retained weights *before* renormalization, dividing every w'_k by $m \cdot S'/n$ to enforce unit mean removes exactly this constant factor from every particle's weight:

$$c = \frac{mS'}{n}.$$

This factor vanishes at $\beta = 0$ (where $S' = \sum_j w_j = n$ and every retained weight is 1, so $m = 1$ and $c = 1$) and at $\beta = 1$ (where $S' = n$ trivially and, since the resampling permutation is the identity, $m = n^{-1} \sum_j w_j = 1$, so again $c = 1$), but for $0 < \beta < 1$ we have $c \neq 1$ in general.

It is tempting to conclude that the running likelihood is therefore understated by a factor of c at every such resampling step unless c is credited back, and that Aaron's Julia routine — which returns **nothing**, its caller discarding it —

Listing 4: Aaron's `pfilt_step_comps!` (PartiallyObservedMarkovProcesses.jl_aarons/src/pfilter.jl, line 228)

```
systematic_resample!(p, w, work, target)    # return value discarded
```

is biased low for $0 < \beta < 1$. That conclusion is wrong, and the reason it is wrong is worth stating carefully, since the argument for it is superficially convincing.

The derivation above establishes that the credited scheme satisfies the proper-weighting identity

$$\mathbb{E} \left[\hat{Z} \cdot \frac{1}{n} \sum_j W_j f(x^j) \right] = \gamma_k(f)$$

exactly, for every test function f and every β . The uncredited scheme does not satisfy *that* identity. But failure of one sufficient condition is not proof of bias: the uncredited scheme satisfies a *different* one — the telescoping identity, under which the normalization applied at step k cancels against the increment formed at step $k + 1$, because each conditional likelihood is a mean of carried weights against the next measurement density and is therefore invariant to a common rescaling of those weights. The two are alternative routes to unbiasedness, not a correct and an incorrect scheme. This is precisely the bookkeeping Aaron's R implementation uses: it leaves the carried weights unnormalized and computes each conditional log-likelihood as a ratio of successive *total* masses (`wfilter.c`, line 110, `loglik = maxlogw + log(w) - log(oldw)`). Note that R credits nothing at resampling either — the mass change across the resampling step is simply never entered into the likelihood, and the next increment is taken relative to the new baseline. The R and Julia versions compute the same estimator.

The question was settled numerically on the linear-Gaussian reduction of the Gompertz model, where the exact likelihood is available in closed form from the Kalman filter. Both estimators were computed from the *same* draws — the particle cloud is identical under the two conventions, which differ only in a scalar accumulator

— so the comparison is exactly paired and the standard error of the difference is far smaller than that of either mean. Over 400,000 replicates at each of $\beta \in \{0.25, 0.5, 0.75\}$ and $n \in \{8, 16, 64\}$, the two expectations agree to within 0.003% of $\hat{L}$, with no significant difference at any setting. Representative rows:

n	β	$\mathbb{E}[\tilde{L}_{\text{cred}}] - \mathbb{E}[\tilde{L}_{\text{uncred}}]$	z
8	0.50	-0.000914 ± 0.000451	-2.02
16	0.50	-0.000175 ± 0.000196	-0.89
64	0.50	-0.000062 ± 0.000061	-1.02

The explanation is that $c = mS'/n$ is a *conditionally mean-one* random factor: since $\mathbb{E}[m] = \sum_j \pi_j w_j^\beta = n/S'$, we have $\mathbb{E}[c] = 1$ exactly. Empirically it is also uncorrelated with the estimate, so multiplying by it changes no expectation and only adds variance — at $n = 8$, $\beta = 0.5$ the multiplier has roughly 10% relative spread.

Debsurya’s translation initially credited the factor explicitly and has since had the credit removed, on exactly these grounds. Both implementations now apply no correction, and the retained weights are simply renormalized to unit mean:

Listing 5: Debsurya’s `systematic_resample!` (PartiallyObservedMarkovProcesses.jl/src/pfilter.jl)

```
w ./= mean(w)    # subsequent steps rely on unit-mean weights
nothing
```

The episode is recorded here rather than quietly deleted because the mistaken argument is a natural one, and because the resolution — that a scheme can be unbiased by a route other than proper weighting — is the substantive point.

4 Terminal ancestry draw under unequal terminal weights

Whenever $\beta > 0$, or a step is skipped by `trigger`, the terminal particle cloud carries unequal weights: the filtering distribution’s information sits partly in the weights, not solely in the particle multiplicities. Recovering a single properly weighted draw of the filtered trajectory therefore requires initiating the traced lineage at the terminal time with probability proportional to the terminal weights, then tracing ancestry backward through the recorded permutations.

Aaron’s `trace_ancestry!` initiates the draw uniformly, regardless of the terminal weights:

Listing 6: Aaron’s `trace_ancestry!` (PartiallyObservedMarkovProcesses.jl_aarons/src/pfilter.jl, line 397)

```
j::I = rand(axes(perm,2))    # uniform draw, regardless of terminal weights
```

A marginal comment in that file (`# FIXME: check that the first index in the ancestry is correct`) suggests this was already a point of concern.

A minimal counterexample makes the resulting bias concrete. Suppose two terminal particles carry weights $W = (0.9, 0.1)$ — an entirely ordinary outcome once $\beta > 0$ or a step goes unresampled. The correct marginal draw of the terminal state assigns particle 1 probability 0.9 and particle 2 probability 0.1, so that a functional such as the traced trajectory’s terminal value has, e.g., $\mathbb{P}(X_T = x^{(1)}) = 0.9$. Aaron’s uniform draw instead assigns each particle probability 0.5 regardless of W , systematically over-weighting the low-probability particle and under-weighting the high-probability one; the traced trajectory is properly weighted for the filtering distribution only by coincidence, when the terminal weights happen to be equal.

Debsurya’s `trace_ancestry!` initiates the draw proportional to the terminal weights before tracing backward:

Listing 7: Debsurya’s weighted `trace_ancestry!` (PartiallyObservedMarkovProcesses.jl/src/pfilter.jl, lines 508–544), initiation step

```
s::W = 0
for k in eachindex(w)
    v::W = w[k]
    s += v
    work[k] = s
end
u::W = s*rand(LogLik)
jj::I = 1
while (u > work[jj] && jj < n)
```

```

    jj += 1
  end
  j = jj
  # ... trace backward from j exactly as before

```

with a fallback to the uniform draw only in the degenerate case where every terminal weight underflows to zero. This routine was exercised in the model-based validation against R `pomp` and the closed-form Gompertz likelihood described elsewhere, and directly addresses the concern flagged in Aaron’s comment.

5 A per-step resampling diagnostic

Debsurya’s `PfilterdPompObject` carries an additional field, `resampled::Array{Bool,1}`, recording at each observation time whether systematic resampling actually occurred there (as opposed to being skipped because ESS stayed above the `trigger` threshold), together with an accessor:

Listing 8: `PartiallyObservedMarkovProcesses.jl/src/pfilter.jl`, lines 29–35

```

"""
    resampled(object)

‘resampled’ extracts the vector of booleans recording, for each
observation time, whether systematic resampling occurred.
"""
resampled(object::PfilterdPompObject) = object.resampled

```

This is set alongside the ancestry permutation in `pfilt_step_comps!` (`resamp[] = true` when resampling occurs, `false` otherwise) and gives a cheap, per-step diagnostic of weight degeneracy over the course of a filtering run, independent of and complementary to the effective sample size trace. Aaron’s `PfilterdPompObject` has no corresponding field.

6 Unification: `wpfilter` as a thin wrapper

In both repositories, the classical bootstrap filter (`trigger = 1`, `target = 0`) and the general trigger/target filter were, at one point, separately implemented engines: a fast path with no weight-carrying overhead for the classical case, and a general path that threads a carried weight vector `wcarry` through the recursion (folded into the incremental log weights at `compute_ess_logLik!`, and consulted at each resampling step). Debsurya’s `pfilter` still dispatches internally between these two engines on the basis of whether `trigger==1 && target==0`, but both are reached through a single public entry point, and `wpfilter` — the name inherited from the R `pomp` package’s separate weighted-filter routine — is now nothing more than a same-signature call-through to `pfilter`:

Listing 9: `PartiallyObservedMarkovProcesses.jl/src/wpfilter.jl`, in full

```

wpfilter(
    object::ValidPompData;
    trigger::Real = 1,
    target::Real = 0,
    kwargs...,
) = pfilter(object; trigger, target, kwargs...)

wpfilter(
    object::PfilterdPompObject;
    Np::Integer = object.Np,
    trigger::Real = object.trigger,
    target::Real = object.target,
    kwargs...,
) = pfilter(object; Np, trigger, target, kwargs...)

wpfilter(_...) = error("Incorrect call to ‘wpfilter’.")

```

There is exactly one particle-filtering implementation to validate, maintain, and reason about; `wpfilter` survives purely as an API-compatible alias, with defaults (`trigger = 1`, `target = 0`) that in fact route it to the same classical engine as a bare call to `pfilter`. This unification is also what makes it possible for `mif2`, the iterated-filtering routine, to run its inner filtering step under `target > 0`: since there is only one filtering routine underneath both names, a weighted filtering step is automatically available wherever a classical one was, which is not presently true of the R implementation, whose weighted filter does not resample parameters.

7 Summary

Issue	Aaron’s Julia	Debsurya’s Julia
Retained weight indexing (§2)	positional: w_k^β	ancestor-indexed: $w_{p(k)}^\beta$
Mass credit at $0 < \beta < 1$ (§3)	none	none <i>(credit added, then reverted: no bias, added variance)</i>
Terminal ancestry draw (§4)	uniform	proportional to terminal weight
Per-step resampling flag (§5)	absent	<code>resampled(object)</code>
<code>wpfilter</code> (§6)	separate historical engine	thin wrapper over <code>pfilter</code>

The first and third rows are not differences of taste: for $\text{target} > 0$ they change, respectively, which particle’s information is retained after resampling and the distribution represented by the stored filtered trajectory. Both coincide with Aaron’s own implementations only at the untempered endpoints $\beta \in \{0, 1\}$, which is presumably why they have not previously been visible in the classical ($\text{target} = 0$) filter that both repositories have exercised most. Neither, however, disturbs the marginal log-likelihood, so neither would have been caught by comparing likelihood estimates: they were found by reading the code. The second row, by contrast, *would* have shown up in such a comparison had it been a genuine defect — and the fact that it did not is exactly the evidence that it was not one.